

1. Project

New Spring Starter Project 

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

Package:

Working sets

☐ Add project to working sets

Working sets:

2. Dependencies

Selected:

- ☒ Spring Boot DevTools
- ☒ Lombok
- ☒ Validation
- ☒ Spring Data JPA
- ☒ MySQL Driver
- ☒ Spring Security
- ☒ Spring Web

3. Create database – petstore in mysql
Create database petstore;
4. Add table details in spring application.properties

```
spring.application.name=PetStore210
spring.datasource.url=jdbc:mysql://localhost:3307/petstore
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

5. Add JWT dependencies in pom.xml (here I am sharing complete pom.xml file)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.0.0</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.tejas</groupId>
  <artifactId>PetStore210</artifactId>
```

```
<version>1.0.0</version>
<name>PetStore210</name>
<description>T210 Final Project</description>
<url/>
<licenses>
  <license/>
</licenses>
<developers>
  <developer/>
</developers>
<scm>
  <connection/>
  <developerConnection/>
  <tag/>
  <url/>
</scm>
<properties>
  <java.version>17</java.version>
</properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

```

    <!-- JWT API -->
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-api</artifactId>
      <version>0.11.5</version>
    </dependency>

    <!-- JWT Implementation -->
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-impl</artifactId>
      <version>0.11.5</version>
      <scope>runtime</scope>
    </dependency>

    <!-- JWT Jackson JSON Parser -->
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-jackson</artifactId>
      <version>0.11.5</version>
      <scope>runtime</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <configuration>
          <annotationProcessorPaths>
            <path>
              <groupId>org.projectlombok</groupId>
              <artifactId>lombok</artifactId>
            </path>
          </annotationProcessorPaths>
        </configuration>
      </plugin>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
        <configuration>
          <excludes>
            <exclude>
              <groupId>org.projectlombok</groupId>
              <artifactId>lombok</artifactId>
            </exclude>
          </excludes>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>

```

6. Create following packages –

```

v PetStore210 [boot] [devtools]
  v src/main/java
    > com.tejas
    > com.tejas.configurations
    > com.tejas.controllers
    > com.tejas.daos
    > com.tejas.enums
    > com.tejas.jwt
    > com.tejas.models
    > com.tejas.repositories
    > com.tejas.responsewrapper
    > com.tejas.services
    > com.tejas.specifications

```

PetType $\leftarrow 1 \ N \rightarrow$ PetBreed (create pet_type column in PetBreed)

Pet (create breed_id column in Pet) $\leftarrow N \quad 1 \rightarrow$ PetBreed

7. Implementing user registration, login and Spring security

- a. Create **Role.java enum** in enums package (right click on package > new > enum)
Role.java

```
public enum Role
{
    CUSTOMER, ADMIN
}
```

- b. Create User.java class in models package
User.java

```
@Data
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;

    private String fullName;

    private String email;

    private String password;

    private String address;

    private String pincode;

    @Enumerated(EnumType.STRING)
    private Role role;
}
```

- c. Create UserRepository.java interface in repositories package

```
@Repository
public interface UserRepository extends JpaRepository<User, Long>
{
    //to get user details by email while login
    Optional<User> findByEmail(String email);
    //to check if email already exist while registration
    boolean existsByEmail(String email);
}
```

- d. Create class MyUserDetailsService.java in services package
MyUserDetailsService.java

```
@Service
public class MyUserDetailsService implements UserDetailsService{
    @Autowired
```

```

        private UserRepository userRepository;

        @Override
        public UserDetails loadUserByUsername(String email) throws
UsernameNotFoundException {
            User user=userRepository.findByEmail(email) //here User is our
model/entity
            .orElseThrow(() ->
                new UsernameNotFoundException("Email not found: " + email)
            );

            List<SimpleGrantedAuthority> authorities =
                List.of(new SimpleGrantedAuthority(user.getRole().name()));

            return new org.springframework.security.core.userdetails.User(
                user.getEmail(),
                user.getPassword(),
                authorities
            );
        }
    }
}

```

e. Create class JWTTokenGenerator.java in jwt package

```

@Component
public class JWTTokenGenerator
{
    private final String SECRET = "GDF7acJj3fHj48U4/rtgWkSb31XKkqv0cHpP9yJJ2kU=";

    // Extract EMAIL from JWT
    public String extractEmail(String token) {
        Claims claims = extractAllClaims(token);
        return claims.get("email", String.class);
    }

    // Extract ROLE from JWT
    public String extractRole(String token) {
        Claims claims = extractAllClaims(token);
        return claims.get("role", String.class);
    }

    // Extract all claims
    private Claims extractAllClaims(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(Keys.hmacShaKeyFor(SECRET.getBytes()))
            .build()

```

```

        .parseClaimsJws(token)
        .getBody();
    }

    // Check if token expired
    private Boolean isExpired(String token) {
        Claims claims = extractAllClaims(token);
        Date expiration = claims.getExpiration();
        return expiration.before(new Date());
    }

    // Validate token
    public Boolean validateToken(String token, UserDetails userDetails) {
        String email = extractEmail(token);
        return (email.equals(userDetails.getUsername()) && !isExpired(token));
    }

    public String generateToken(UserDetails userDetails, String role) {
        System.out.println(role);
        Map<String, Object> claims = new HashMap<>();
        claims.put("email", userDetails.getUsername()); // username = email
        claims.put("role", role);

        return Jwts.builder()
            .setClaims(claims)
            .setSubject(userDetails.getUsername()) // optional
            .setIssuedAt(new Date()) // below calculation is for 10 hrs
            .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 *
60 * 10))
            .signWith(Keys.hmacShaKeyFor(SECRET.getBytes()),
SignatureAlgorithm.HS256)
            .compact();
    }
}

```

- f. Create class JWTRequestValidator.java in jwt package
JWTRequestValidator.java

```

@Component
public class JWTRequestValidator extends OncePerRequestFilter {

    @Autowired
    JWTTokenGenerator jwtTokenGenerator;

    @Autowired
    MyUserDetailsService myUserDetailsService;

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain)
    {
    }
}

```

```

        throws ServletException, IOException {

        final String header = request.getHeader("Authorization");
        String jwtToken = null;
        String email = null;
        String role = null;

        // Extract token
        if (header != null && header.startsWith("Bearer "))
        {
            jwtToken = header.substring(7);

            try
            {
                email = jwtTokenGenerator.extractEmail(jwtToken);
                role = jwtTokenGenerator.extractRole(jwtToken);
            } catch (Exception e)
            {
                System.out.println("Invalid Token: " + e.getMessage());
            }
        }

        // Authenticate user if token is valid
        if (email != null && SecurityContextHolder.getContext().getAuthentication() ==
null) {

            UserDetails userDetails = myUserDetailsService.loadUserByUsername(email);

            if (jwtTokenGenerator.validateToken(jwtToken, userDetails)) {

                List<SimpleGrantedAuthority> authorities =
                    List.of(new SimpleGrantedAuthority(role));

                System.out.println("ROLE FROM TOKEN = " + role);
                System.out.println("AUTHORITIES SET = " + authorities);

                UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(
                        userDetails,
                        null,
                        authorities
                    );

                authToken.setDetails(
                    new WebAuthenticationDetailsSource().buildDetails(request)
                );

                SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }

        filterChain.doFilter(request, response);
    }
}

```

g. Create SecurityConfiguration.java class in configurations package

```

@Configuration
@EnableWebSecurity
public class SecurityConfiguration {

    @Autowired
    private MyUserDetailsService userDetailsService;

```

```

@Autowired
private JWTTokenGenerator jwtTokenGenerator;

@Autowired
private JWTRequestValidator jwtRequestValidator;

@Bean
PasswordEncoder myPasswordEncoder()
{
    return new BCryptPasswordEncoder();
}

@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration
configuration) throws Exception {
    return configuration.getAuthenticationManager();
}

@Bean
SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity) throws Exception
{
    httpSecurity
        .cors(cors-> cors.configurationSource(corsConfigurationSource()))
        .csrf(anyRequest -> anyRequest.disable()) // disable CSRF
        .authorizeHttpRequests(request -> request
            .requestMatchers("/api/v1/login",
"/api/v1/register").permitAll()
            .requestMatchers("/api/v1/get/**").permitAll()
            .requestMatchers("/api/v1/filter").permitAll()
            .requestMatchers("/images/**").permitAll()
            .requestMatchers("/api/v1/admin/**").hasAuthority("ADMIN")
            .anyRequest().authenticated())
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        );
    httpSecurity.addFilterBefore(jwtRequestValidator,
UsernamePasswordAuthenticationFilter.class);
    return httpSecurity.build();
}

//below must be imported as -
//import org.springframework.web.cors.CorsConfiguration; NOT FROM REACTIVE
//import org.springframework.web.cors.CorsConfigurationSource;
//import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of("http://localhost:3000")); // React app
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "PATCH",
"OPTIONS"));
    config.setAllowedHeaders(List.of("")); // allow Authorization header
    config.setExposedHeaders(List.of(""));
    config.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}
}

```

- h. Create UserLogin.java class in daos package

```
@Data
public class UserLogin {
    private String email;
    private String password;
}
```

- i. Create UserRegister.java class in daos package

```
@Data
public class UserRegister {
    private String email;
    private String password;
    private Role role;
}
```

- j. Create MyResponseWrapper.java class in responsewrapper package

```
@Component
@Data
public class MyResponseWrapper
{
    private String message;
    private Object data;
}
```

- k. Create JwtResponseWrapper.java class in responsewrapper package

```
@Data
@Component
public class JwtResponseWrapper
{
    private String token;
    private String role;
    private long userId;
}
```

- l. Create UserService.java class in services package

```
@Service
public class UserService {
    @Autowired
    PasswordEncoder passwordEncoder;

    @Autowired
    MyResponseWrapper responseWrapper;

    @Autowired
    UserRepository userRepository;
    private ResponseEntity<> universalResponse(String message, Object data, HttpStatus
httpStatus)
    {
        responseWrapper.setMessage(message);
        responseWrapper.setData(data);
        return new ResponseEntity<> (responseWrapper,httpStatus);
    }

    public ResponseEntity<> register(UserRegister userRegister)
    {
        if(userRepository.existsByEmail(userRegister.getEmail()))
```

```

        {
            return universalResponse("Email Exists",null,
HttpStatus.INTERNAL_SERVER_ERROR);
        }
        User newUser = new User();
        newUser.setEmail(userRegister.getEmail());
        newUser.setPassword(passwordEncoder.encode(userRegister.getPassword()));
        newUser.setRole(userRegister.getRole());
        User registeredUser=userRepository.save(newUser);
        return universalResponse("Registration Success!",registeredUser,
HttpStatus.CREATED);
    }

    @Autowired
    private AuthenticationManager authenticationManager;

    @Autowired
    private JWTTokenGenerator jwtTokenGenerator;

    @Autowired
    private MyUserDetailsService myUserDetailsService;

    public ResponseEntity<?> login(UserLogin userLogin)
    {
        try
        {
            authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(userLogin.getEmail(),
userLogin.getPassword())
            );
        }
        catch (BadCredentialsException e)
        {
            return universalResponse("Incorrect Username or password",null,
HttpStatus.BAD_REQUEST);
        }

        User user =userRepository.findByEmail(userLogin.getEmail()).get();
        UserDetails userDetails =
myUserDetailsService.loadUserByUsername(userLogin.getEmail());
        String jwtToken =jwtTokenGenerator.generateToken(userDetails,
user.getRole().toString());

        JwtResponseWrapper jwtResponseWrapper = new JwtResponseWrapper();
        jwtResponseWrapper.setToken(jwtToken);
        jwtResponseWrapper.setRole(user.getRole().name());
        jwtResponseWrapper.setUserId(user.getId());
        return universalResponse("Login Success",jwtResponseWrapper, HttpStatus.OK);
    }
}

```

m. Create UserController.java class in controllers package

```

@RestController
@RequestMapping("/api/v1")
public class UserController
{

    @Autowired
    UserService userService;
}

```

```

@PostMapping("/register")
public ResponseEntity<?> register(@RequestBody UserRegister userRegister)
{
    return userService.register(userRegister);
}

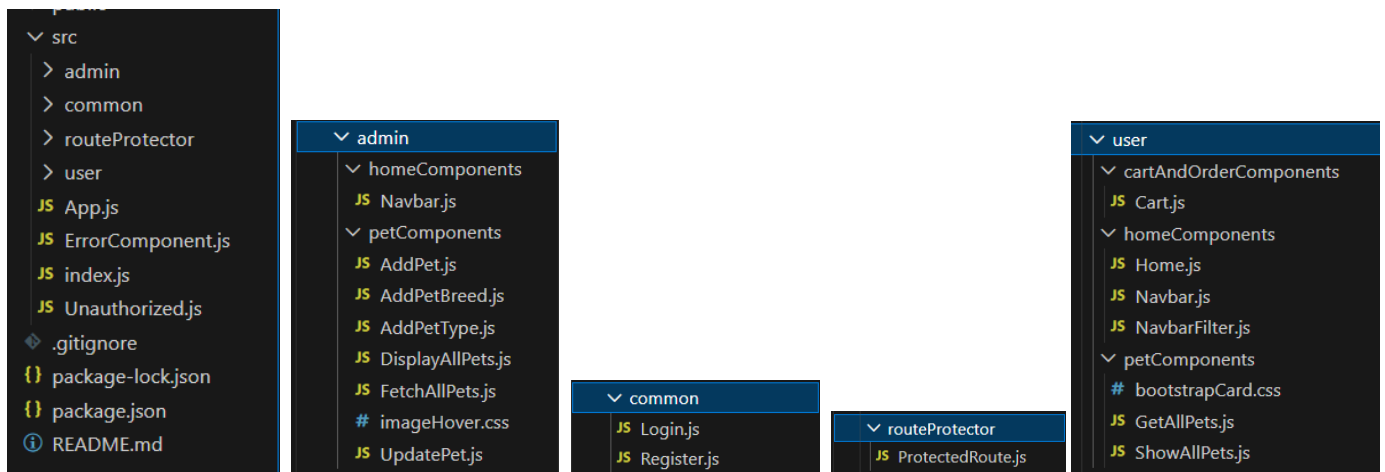
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody UserLogin userLogin)
{
    return userService.login(userLogin);
}
}

```

n. Backend DONE!! - run spring server

8. Create React project and add following dependencies –

- a. >npx create-react-app pet-store
- b. >cd pet-store
- c. pet-store>
 - i. bootstrap
 - ii. react hook form
 - iii. bootstrap icons
 - iv. react router dom
- d. open project in VS code and create following project structure



Add rfc code in all components

- e. create protected routes (role based access in routeProtector/ProtectedRoute.js)

```

import { Navigate, Outlet } from "react-router-dom";

const ProtectedRoute = ({ roleRequired }) => {
  // const token = localStorage.getItem("token");
  // const role = localStorage.getItem("role");

  const user = JSON.parse(localStorage.getItem("user")); // { role: "ADMIN" or "CUSTOMER" }

  // not logged in
  if (!user) {
    return <Navigate to="/login" replace />;
  }

  // wrong role
  if (user.role !== roleRequired) {
    return <Navigate to="/unauthorized" replace />;
  }

  // render child routes
  return <Outlet />;
}

```

```
});  
export default ProtectedRoute;
```

f. code for src/ErrorComponent.js

```
import React from 'react'  
  
export default function ErrorComponent() {  
  return (  
    <div>  
      <h1>404 This page not exists!</h1>  
    </div>  
  )  
}
```

g. code for src/Unauthorized.js

```
import React from 'react'  
  
export default function Unauthorized() {  
  return (  
    <div>  
      Unauthorized  
    </div>  
  )  
}
```

h. create routes for all components in index.js

```
import '../node_modules/bootstrap/dist/css/bootstrap.min.css'  
import '../node_modules/bootstrap/dist/js/bootstrap.min.js'  
import '../node_modules/bootstrap-icons/font/bootstrap-icons.min.css'  
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import { createBrowserRouter, RouterProvider } from 'react-router-dom';  
import App from './App';  
import ErrorComponent from './ErrorComponent';  
import AddPetType from './admin/petComponents/AddPetType';  
import AddPetBreed from './admin/petComponents/AddPetBreed';  
  
import AddPet from './admin/petComponents/AddPet.js';  
import UpdatePet from './admin/petComponents/UpdatePet.js';  
import FetchAllPets from './admin/petComponents/FetchAllPets.js';  
import Home from './user/homeComponents/Home.js';  
import Unauthorized from './Unauthorized.js';  
import Login from './common/Login.js';  
import Register from './common/Register.js';  
import ProtectedRoute from './routeProtector/ProtectedRoute.js';  
import GetAllPets from './user/petComponents/GetAllPets.js';  
import Cart from './user/cartAndOrderComponents/Cart.js';  
  
const allRoutes=createBrowserRouter([  
  //new routing after spring security  
  //ADMIN routes  
  {  
    path:'/admin',  
    element:<ProtectedRoute roleRequired="ADMIN"/>,  
    children:  
    [  
      {  
        path:"",
```

```

        element:<App/>,
        errorElement:<ErrorComponent/>,
        children:[
            { path: "add-pet-type", element: <AddPetType /> },
            { path: "add-pet-breed", element: <AddPetBreed /> },
            { path: "add-pet", element: <AddPet /> },
            { path: "manage-pets", element: <FetchAllPets /> },
            { path: "update-pet/:petId", element: <UpdatePet /> }
        ]
    }
],
},
// PUBLIC ROUTES
{
    path: "/",
    element: <Home />,
    errorElement: <ErrorComponent />,
    children:
    [
        { path: "", element: <GetAllPets /> },
        { path: "login", element: <Login /> },
        { path: "register", element: <Register /> },
        { path: "unauthorized", element: <Unauthorized /> }, // Create a simple
unauthorized page,
        {
            path: "my",
            element: <ProtectedRoute roleRequired="CUSTOMER" />, // * Customer protected
            children:
            [
                { path: "cart", element: <Cart/> }
            ]
        },
    ],
}
])
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
    <RouterProvider router={allRoutes}/>
);

```

9. implementing login and register functionality
 - a. code for src/common/Login.js

```

import React from 'react'

import { useForm } from "react-hook-form";
import { useNavigate } from "react-router-dom";

export default function Login() {
    let { register, handleSubmit, formState } = useForm();

```

```

const navigate = useNavigate();

async function handleLogin (formData)
{
  console.log(formData);
  let response = await fetch(
    `http://localhost:8080/api/v1/login`,
    {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(formData),
    }
  );
  let responseObject = await response.json();
  localStorage.setItem("user", JSON.stringify(responseObject.data));
  //console.log(responseObject);
  // Navigate based on role
  const role = responseObject.data.role;

  if (role === "ADMIN") {
    navigate("/admin");
  } else if (role === "CUSTOMER") {
    navigate("/");
  } else {
    navigate("/");
  }
};

return (
  <div className='d-flex flex-column align-items-center'>
    <h1>Login!</h1>
    <form className='w-50' onSubmit={handleSubmit(handleLogin)}>

      <table className="table table-hover table-bordered table-dark">
        <tbody>
          <tr>
            <td> <label for="email" class="form-label">Email</label> </td>

```

```

        <td> <input type="email" class="form-control" id="email"
            {...register('email',{required:true,pattern:/^[^\s@]+@[^\s@]+\.[^\s@]+$/})
        )}/></td>

    </tr>
    <tr>
        <td> <label for="password" class="form-label">Password</label> </td>
        <td> <input type="password" class="form-control" id="password"
            {...register('password',{required:true})}/></td>
    </tr>
    <tr>
        <td colspan={2}> <button type="submit" className="btn btn-primary w-
100">Submit</button></td>
    </tr>
</tbody>
</table>
</form>
</div>
)
}

```

b. code for src/common/Register.js

```

import React from 'react'
import { useForm } from 'react-hook-form';
import { replace, useNavigate } from 'react-router-dom';

export default function Register() {
    let { register, handleSubmit, formState } = useForm();
    const navigate = useNavigate();

    async function handleRegister (formData)
    {
        console.log(formData);
        let response = await fetch(
            `http://localhost:8080/api/v1/register`,
            {
                method: "POST",
                headers: { "Content-Type": "application/json" },
            }
        )
    }
}

```



```

        </tr>

        <tr>
            <td colSpan={2}> <button type="submit" className="btn btn-primary w-
100">Submit</button></td>
        </tr>
    </tbody>

</table>
</form>
</div>
)
}

```

c. code for user/homeComponents/Home.js

```

import Navbar from './Navbar'
import { Outlet } from 'react-router-dom'

export default function Home() {
    return (
        <div>
            <Navbar/>
            <Outlet/>
        </div>
    )
}

```

d. code for user/homeComponents/Navbar.js

```

import React from 'react'
import { Link, useNavigate } from 'react-router-dom'

export default function Navbar() {
    const user = JSON.parse(localStorage.getItem("user"));
    const navigate = useNavigate();

    const handleLogout = () => {

```

```

localStorage.removeItem("user");
navigate("/");
};

return (
  <div>
    <nav className="navbar navbar-expand-lg bg-dark" data-bs-theme="dark">
      <div className="container-fluid">
        <Link className="navbar-brand" to={'/'}><i className="bi bi-shop"></i> &nbsp; Pet-
Store</Link>
        <button className="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarSupportedContent" aria-controls="navbarSupportedContent" aria-expanded="false"
aria-label="Toggle navigation">
          <span className="navbar-toggler-icon"></span>
        </button>
        <div className="collapse navbar-collapse" id="navbarSupportedContent">
          <ul className="navbar-nav me-auto mb-2 mb-lg-0">
            <li className="nav-item">
              <Link className="nav-link active" aria-current="page" to={'/'}>Home</Link>
            </li>
            <li className="nav-item">
              <Link className="nav-link active" aria-current="page" to={'/'}>Pets</Link>
            </li>
            {user && (
              <li className="nav-item">
                <a className="nav-link active" aria-current="page" href="#">Cart</a>
              </li>
              <li className="nav-item">
                <a className="nav-link active" aria-current="page" href="#">Orders</a>
              </li>
            )}
          </ul>
        </div>
      </div>
    </nav>
  )
);

```

```

        {!user && (
          <>
            <li className="nav-item">
              <Link className="nav-link active" aria-current="page"
to={'/register'}>Register</Link>
            </li>
            <li className="nav-item">
              <Link className="nav-link active" aria-current="page"
to={'/login'}>Login</Link>
            </li>
          </>
        )}

        {user && (
          <>
            <li className="nav-item">
              <button className="nav-link btn btn-link"
onClick={handleLogout}>Logout</button>
            </li>
          </>
        )}
      </ul>
    </div>
  </div>
</nav>
</div>
)
}

```

- e. after this, run react project and perform register and Login(**TESTED! WORKING!!!!!!**)
- f. DONEEE!!!!!!!!!!

10. Creating API to add PetType and get all pet types (Both backend and frontend code)

- a. Create PetType.java class in models package

```
@Entity
@Data
public class PetType
{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;

    @Column(unique = true, nullable = false)
    private String typeName;
}
```

- b. Create PetTypeRepository.java interface in repositories package

```
@Repository
public interface PetTypeRepository extends JpaRepository<PetType, Long>
{
}
}
```

- c. Create PetTypeService.java class in services package

```
@Service
public class PetTypeService {

    @Autowired
    PetTypeRepository petTypeRepository;

    @Autowired
    MyResponseWrapper responseWrapper;

    private ResponseEntity<?> universalResponse(String message, Object data, HttpStatus httpStatus)
    {
        responseWrapper.setMessage(message);
        responseWrapper.setData(data);
        return new ResponseEntity<> (responseWrapper, httpStatus);
    }

    public ResponseEntity<?> addPetType(PetType petType)
    {
        if(petType.getTypeName().length()<3 || petType == null)
        {
            return universalResponse("Please enter valid pet Type", null, HttpStatus.BAD_REQUEST);
        }else {
            PetType savedPetType = petTypeRepository.save(petType);
            return universalResponse("Pet type added successfully", savedPetType, HttpStatus.OK);
        }
    }

    public ResponseEntity<?> getAllPetTypes()
    {
        List<PetType> petTypes= petTypeRepository.findAll();
        if(petTypes.size()==0)
        {
            return universalResponse("There are no pet types found", null, HttpStatus.NOT_FOUND);
        }else {
            return universalResponse("List of All pet types", petTypes, HttpStatus.FOUND);
        }
    }
}
```

d. Create `PetTypeController.java` class in controllers package

```
@RestController
@RequestMapping("/api/v1")
public class PetTypeController
{
    @Autowired
    PetTypeService petTypeService;

    //only accessible by ADMIN
    @PostMapping("/admin/add-pet-type")
    public ResponseEntity<?> addPetType(@RequestBody PetType petType)
    {
        return petTypeService.addPetType(petType);
    }

    //public API
    @GetMapping("/get/pet-types")
    public ResponseEntity<?> getAllPetTypes()
    {
        return petTypeService.getAllPetTypes();
    }
}
```

e. In react project, code for App.js in src/App.js

```
import React from 'react'
import Navbar from './admin/homeComponents/Navbar'
import { Outlet } from 'react-router-dom'

export default function App() {
  return (
    <div>
      <Navbar/>
      <Outlet/>
    </div>
  )
}
```

f. In react project, code Navbar.js in src/admin/homeComponents/Navbar.js

```
import React from 'react'
import { Link } from 'react-router-dom'

export default function Navbar() {
  const handleLogout = () => {
    localStorage.removeItem("user"); // remove token + role + userId
    window.location.href = "/login"; // hard redirect to clean memory
  };

  return (
    <div>
      <nav className="navbar navbar-expand-lg bg-dark" data-bs-theme="dark">
        <div className="container-fluid">
          <Link className="navbar-brand" to={'/admin'}><i className="bi bi-shop"></i> &nbsp; Pet-
Store</Link>
          <button className="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarSupportedContent" aria-controls="navbarSupportedContent" aria-expanded="false" aria-
label="Toggle navigation">
            <span className="navbar-toggler-icon"></span>
          </button>
          <div className="collapse navbar-collapse" id="navbarSupportedContent">
            <ul className="navbar-nav me-auto mb-2 mb-lg-0">
              {/* <li className="nav-item">
                <a className="nav-link active" aria-current="page" href="#">Home</a>
              </li> */}
              <li className="nav-item dropdown">
                <a className="nav-link dropdown-toggle" href="#" role="button" data-bs-toggle="dropdown"
aria-expanded="false">
                  Pets
                </a>
                <ul className="dropdown-menu">
```

```

        <li><Link className="dropdown-item" to={'/admin/add-pet-type'}>Add Pet Type</Link></li>
        <li><hr className="dropdown-divider"/></li>
        <li><Link className="dropdown-item" to={'/admin/add-pet-breed'}>Add Pet
Breed</Link></li>
        <li><hr className="dropdown-divider"/></li>
        <li><Link className="dropdown-item" to={'/admin/add-pet'}>Add Pet</Link></li>
        <li><hr className="dropdown-divider"/></li>
        <li><Link className="dropdown-item" to={'/admin/manage-pets'}>Manage Pets</Link></li>
    </ul>
</li>
</ul>
</div>
<div>
    <ul className="navbar-nav me-auto mb-2 mb-lg-0">

        <li className="nav-item">
            {/* <a className="nav-link active" aria-current="page" href="#">Logout</a> */}
            <button className="nav-link btn btn-link" onClick={handleLogout}>
                Logout
            </button>
        </li>

    </ul>
</div>
</div>
</nav>
</div>
)}}

```

g. In react project, code AddPetType.js in in src/admin/petComponents/AddPetType.js

```

import React, { useEffect, useState } from 'react'
import { useForm } from 'react-hook-form'

export default function AddPetType() {

    let {register, handleSubmit, formState} = useForm();
    let [pets, setPets] = useState(null);
    let [isPetAdded, setIsPetAdded] = useState(false);
    const user = JSON.parse(localStorage.getItem("user"));
    const token = user?.token;

    let submitForm = async (formData) => {
        console.log(formData);
        let response = await fetch('http://localhost:8080/api/v1/admin/add-pet-type',{
            method:'POST',
            headers:{'Content-Type':'application/json','Authorization': `Bearer ${token}`},
            body:JSON.stringify(formData)
        })
        let responseObject = await response.json();
        console.log(responseObject);
        setIsPetAdded(true); //to fetch all pets and display below
    }

    async function fetchAllPetTypes() {
        setIsPetAdded(false)
        let response = await fetch("http://localhost:8080/api/v1/get/pet-types")
        let responseObject = await response.json()
        let allPetTypes = responseObject.data
        // console.log(allPetTypes)
        setPets(allPetTypes)
    }
    // fetchAllPetTypes();
    useEffect(()=>{fetchAllPetTypes()}, [isPetAdded])

    return (
        <div>
            <h3 className='text-center mt-3'>Add Pet Type</h3>
            <form className='d-flex justify-content-center mt-3 border-bottom border-primary border-3 rounded'
onSubmit={handleSubmit(submitForm)}>
                <div className="mb-3 me-3 w-25">

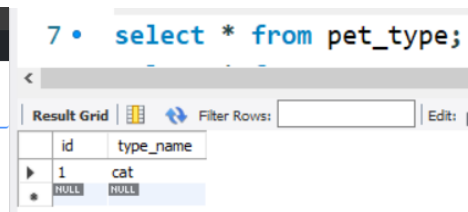
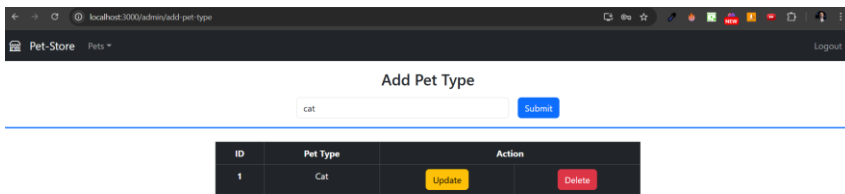
```

```

    <input type="text" className="form-control" placeholder='Enter Pet Type here'
    {...register('typeName',{required:true, minLength:3, maxLength:10})}/>
  <div className='text-danger'>
    {formState.errors.typeName && formState.errors.typeName.type=="required" && "Pet Type is
Required"}
    {formState.errors.typeName && formState.errors.typeName.type=="minLength" && "Pet Type must
have min 3 characters"}
    {formState.errors.typeName && formState.errors.typeName.type=="maxLength" && "Pet Type is not
allowed more than 10 characters"}
  </div>
</div>
<div>
  <button type="submit" className="btn btn-primary">Submit</button>
</div>
</form>
<div className='mt-4 text-center d-flex justify-content-center'>
  <table className="table table-hover table-bordered table-dark w-50">
    <thead>
      <tr>
        <th scope="col">ID</th>
        <th scope="col">Pet Type</th>
        <th scope="col" colSpan={2}>Action</th>
      </tr>
    </thead>
    <tbody>
      {
        pets && pets.map(pet => {
          return <tr key={pet.id}>
            <th scope="row">{pet.id}</th>
            <td>{pet.typeName.toLowerCase()}</td> <td>{pet.typeName.toUpperCase().concat(pet.typeName.slice(1))}</td> <td>{pet.typeName[0].toUpperCase() + pet.typeName.slice(1)}</td>
            <td>
              <button type="button" className="btn btn-warning">Update</button>
            </td>
            <td>
              <button type="button" className="btn btn-danger">Delete</button>
            </td>
          </tr>
        })
      }
    </tbody>
  </table>
</div>
)
}

```

- h. Run spring and react project and check output
 - i. After running spring project, table : pet_type created (checked!!!)
 - ii. Posting pet type from react also working file (I also checked table)



I also

added dog after cat but not added in screenshot

11. Creating API to add PetBreed and get All pet breeds (both in spring and angular)

a. Important :

- i. Relationship between PetType $\leftarrow 1$ $N \rightarrow$ PetBreed (one pet type has multiple pet breed)
- ii. We are going to some extra functionalities while adding a pet breed
 1. Select pet type (from drop down) for which we are adding pet breed
 2. Displaying added pet breed after adding
 3. Filter option to filter pet breed according to pet type

b. Create PetBreed.java class in models package

```
@Entity
@Data
public class PetBreed
{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;

    @Column(nullable = false, unique = true)
    private String breedName;

    @ManyToOne
    @JoinColumn(name = "pet_type", nullable = false)
    private PetType petType;
}
```

c. Create PetBreedRepository.java interface in repositories package

```
@Repository
public interface PetBreedRepository extends JpaRepository<PetBreed, Long>
{
    //JPA method to get pet type based on pet type id
    List<PetBreed> findByPetType_Id(long petTypeId);
}
```

d. Create PetBreedService.java class in services package

```
@Service
public class PetBreedService
{

    @Autowired
    PetBreedRepository petBreedRepository;

    @Autowired
    PetTypeRepository petTypeRepository;

    @Autowired
    MyResponseWrapper responseWrapper;

    private ResponseEntity<?> universalResponse(String message, Object data, HttpStatus httpStatus)
    {
        responseWrapper.setMessage(message);
        responseWrapper.setData(data);
        return new ResponseEntity<> (responseWrapper, httpStatus);
    }

    public ResponseEntity<?> addBreedForPet(PetBreed petBreed, long petTypeId) {
```

```

        Optional<PetType> petType = petTypeRepository.findById(petTypeId);
        if(petType.isPresent()) {
            petBreed.setPetType(petType.get());
            PetBreed savedPetBreed = petBreedRepository.save(petBreed);
            return universalResponse("Pet Breed added successfully", savedPetBreed,
HttpStatus.OK);
        } else {
            return universalResponse("There is no pet found by given pet id", null,
HttpStatus.NOT_FOUND);
        }
    }

    public ResponseEntity<?> getAllPetBreeds() {
        List<PetBreed> allBreeds = petBreedRepository.findAll();

        if (allBreeds.isEmpty()) {
            return universalResponse(
                "No Pet Breeds found",
                null,
                HttpStatus.NOT_FOUND
            );
        } else {
            return universalResponse(
                "Following Pet Breeds found",
                allBreeds,
                HttpStatus.FOUND
            );
        }
    }

    public ResponseEntity<?> getPetBreedsByPetTypeId ( Long petTypeId ){
        List<PetBreed> filteredPetBreeds = petBreedRepository.findByPetType_Id(petTypeId);

        if(filteredPetBreeds.isEmpty()) {
            return universalResponse("There are no Pet breeds found ", null,
HttpStatus.NOT_FOUND);
        }

        else {

            return universalResponse("Following breeds found", filteredPetBreeds,
HttpStatus.FOUND);
        }

    }
}

```

e. Create PetBreedController.java class in controllers package

```

@RestController
@RequestMapping("/api/v1")
public class PetBreedController
{
    @Autowired
    PetBreedService petBreedService;

    //only accessible by ADMIN
    @PostMapping("/admin/add-pet-breed/{petTypeId}")
    public ResponseEntity<?> addBreedForPet(@RequestBody PetBreed petBreed, @PathVariable("petTypeId")
long petTypeId)
    {
        return petBreedService.addBreedForPet(petBreed, petTypeId);
    }

    //Public API
    @GetMapping("/get/pet-breeds")
    public ResponseEntity<?> getAllPetBreeds()
    {
        return petBreedService.getAllPetBreeds();
    }
}

```

```

    }

    @GetMapping("/admin/findBreedByPetTypeId/{petTypeId}")
    public ResponseEntity<?> getPetBreedsByPetTypeId(@PathVariable("petTypeId") long petTypeId)
    {
        return petBreedService.getPetBreedsByPetTypeId(petTypeId);
    }
}

```

f. Code for AddPetBreed.js in src/admin/petComponents/AddPetBreed.js

```

import React, { useEffect, useState } from 'react'
import { useForm } from 'react-hook-form'

export default function AddPetBreed() {
    let { register, handleSubmit, formState } = useForm();
    let [pets, setPets] = useState(null); // list of pet types
    let [breeds, setBreeds] = useState(null); // list of breeds
    let [isPetBreedAdded, setIsPetBreedAdded] = useState(false);

    const user = JSON.parse(localStorage.getItem("user"));
    const token = user?.token;

    let submitForm = async (formData) => {
        console.log(token);

        console.log(formData);
        // formData = {petTypeId: '1', breedName: 'persian'}

        let response = await fetch(
            `http://localhost:8080/api/v1/admin/add-pet-breed/${formData.petTypeId}`,
            {
                method: "POST",
                headers: { "Content-Type": "application/json", "Authorization": `Bearer ${token}` },
                body: JSON.stringify(formData),
            }
        );
        let responseObject = await response.json();
        console.log(responseObject);
        setIsPetBreedAdded(true);
    };

    async function fetchAllPetBreeds() {
        setIsPetBreedAdded(false);
        let response = await fetch("http://localhost:8080/api/v1/get/pet-breeds");
        let responseObject = await response.json();
        let allBreeds = responseObject.data;
        setBreeds(allBreeds);
    }

    async function fetchAllPetTypes() {
        let response = await fetch(
            "http://localhost:8080/api/v1/get/pet-types"
        );
        let responseObject = await response.json();
        let allPetTypes = responseObject.data;
        setPets(allPetTypes);
    }

    async function filterBreedByPetType(event) {
        let petTypeId = event.target.value;
        if (petTypeId == 0) {
            fetchAllPetBreeds();
        } else {
            let response = await fetch(
                `http://localhost:8080/api/v1/admin/findBreedByPetTypeId/${petTypeId}`,
                {
                    headers: {"Authorization": `Bearer ${token}`}
                }
            );
            let responseObject = await response.json();
            let filteredBreeds = responseObject.data;
            setBreeds(filteredBreeds);
        }
    }

    useEffect(() => { fetchAllPetTypes(); }, [])
    useEffect(() => {

```



```

</thead>
<tbody>
  {breeds &&
    breeds.map((breed) => (
      <tr key={breed.id}>
        <th scope="row">{breed.id}</th>
        <td>{breed.breedName.toLowerCase()}</td>
        <td>
          <button type="button" className="btn btn-warning">
            Update
          </button>
        </td>
        <td>
          <button type="button" className="btn btn-danger">
            Delete
          </button>
        </td>
      </tr>
    ))
  }
</tbody>
</table>
</div>
</div>
);
}

```

- g. Run spring and react server and check output
- After running spring server, I checked, pet_breed table is created!
 - After running react server I added pet breed –



Select pet Type: All

ID	BREED	Action
1	persian	Update Delete



Select pet Type: All

ID	BREED	Action
1	persian	Update Delete
2	bulldog	Update Delete

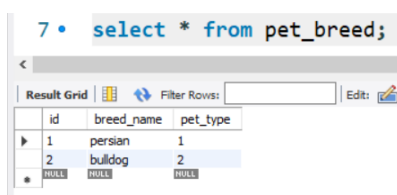
Filtration also working fine

Select pet Type: cat

ID	BREED	Action
1	persian	Update Delete

Select pet Type: dog

ID	BREED	Action
2	bulldog	Update Delete



EVERYTHING WORKING FINE

- DONE EEEEEEEEEEEEEEEEEEEEEEE

